

```
# Data Handling
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Ignore Warnings
import warnings
warnings.filterwarnings("ignore")

# Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

print("All Libraries Imported Successfully.")
```

All Libraries Imported Successfully.

```
df = pd.read_csv("/content/drive/MyDrive/DATA SET /cleaned_merged_dataset_v3.csv")

print("Dataset Loaded Successfully")
print(df.shape)
df.head()
```

Dataset Loaded Successfully
(4020, 83)

	shipment_id	booking_id	customer_id	product_id	carrier_id	warehouse_id	route_id	vehicle_id	customs_id
0	2fbc2ba7-78db-4cd4-abc4-8484f8e296c0	BK100000	9d07a288-d20c-4632-be32-a80646174f82	7d9bcd2f-1cf5-4fdd-ba64-30b5ab5770fe	1b55fff6-046a-4456-9cb0-b8cd21addf5a	c8447ce1-fe0b-4d5b-96c9-b9a275ad8a25	aa747821-06b3-495f-b000-1b2e8b911707	f5ca04c7-97ce-4489-bfc9-69bb578f9273	de1b0e23-43fb-4d0d-a84a-07337bbc6411
1	1881d036-771b-4eb6-ae7a-ac78f41dfd79	BK100001	189fab37-8b05-4e69-b000-be0f6ec9dbc4	5d909ac1-2fad-4861-a19e-79e99e125dfe	2a4ec8f6-16fe-4936-ac67-37fc52238e51	632799e1-4e31-4ba3-838e-c12663328395	1a5d3872-1ddf-4ff8-93e7-07fc5acafc25	830c2f7a-afba-4df9-b8b9-b5e5e1489cee	1d81cb99-8755-4f4a-bc2e-1bb414bbbe42
2	db4bcd2c-d981-4b8a-b61c-83c6dae109d8	BK100002	32320649-068b-4887-b776-ab2d9941cb1a	b1eb6a65-e462-4da5-a91f-78d00b0c241f	22c1d43a-dc69-48e3-8df1-18788f0cc19a	931ea697-c860-4af1-a6c8-e86f72c220ba	ddfeb7c4-87b1-4916-b523-22df1f1c16f3	a33fbf71-6b09-4246-921e-dd0de3c4600f	7e7b4406-6108-4603-897f-0f852c9f8927
3	b0787b6f-8743-4765-a0e8-f0457edad4bf	BK100003	f9670081-1287-4dca-9d8c-4beffacc6dcc	02c94f60-6dc8-4db8-94db-866ee6e7e608	28d939e1-9282-48ee-9a14-a7290d7e0350	a73aac58-381d-4494-84d2-de0456a1c93b	707f01ba-3ee5-4157-9845-41835b01dd03	55e1305e-cd49-4476-8b90-0ab1b3ef5e95	f7b4e378-7c54-45c1-8301-6911110a9b7e
4	473f319b-162e-4a31-a597-a386128b876a	BK100004	d7588951-ab24-43e4-87e7-78cf554e0eb1	a6a78e7a-331b-4e49-9a44-3040e0b2d655	75eda72c-e34b-40aa-b112-e56562058a14	57364895-d2f2-4a7e-9cd0-c2a5050d5b10	2db5db40-7365-4756-9bb7-29e3207a418d	7df258b3-189f-499d-b532-f6a016a22897	5b115244-608a-4e96-98a7-92f5acff7b54

5 rows × 83 columns

```
print("Shape :", df.shape)

print("\nColumns :")
print(df.columns.tolist())

print("\nData Types")
print(df.dtypes)
```

```
print("\nMissing Values")
print(df.isnull().sum())
```

```
Shape : (4020, 83)
```

```
Columns :
['shipment_id', 'booking_id', 'customer_id', 'product_id', 'carrier_id', 'warehouse_id', 'route_id', 'vehicle_id', 'customs_id',
```

```
Data Types
shipment_id    object
booking_id     object
customer_id    object
product_id     object
carrier_id     object
...
temperature    float64
rainfall       float64
humidity       float64
wind_speed     float64
visibility     float64
Length: 83, dtype: object
```

```
Missing Values
shipment_id    0
booking_id     0
customer_id    0
product_id     0
carrier_id     0
..
temperature    0
rainfall       0
humidity       0
wind_speed     0
visibility     0
Length: 83, dtype: int64
```

```
date_columns = [
    'booking_date',
    'ship_date',
    'expected_delivery_date',
    'actual_delivery_date',
    'registration_date',
    'forecast_date'
]

for col in date_columns:
    if col in df.columns:
        df[col] = pd.to_datetime(df[col])

print("Date Conversion Completed.")
```

```
Date Conversion Completed.
```

```
# Target Creation

df['shipment_delay'] = (
    df['actual_delivery_date'] -
    df['expected_delivery_date']
).dt.days

df['delay_days'] = df['shipment_delay'].clip(lower=0)

df['shipment_delayed'] = (
    df['shipment_delay'] > 0
).astype(int)

print(df[['shipment_delay', 'delay_days', 'shipment_delayed']].head())
```

	shipment_delay	delay_days	shipment_delayed
0	3	3	1
1	1	1	1
2	0	0	0
3	3	3	1
4	0	0	0

```
# Booking Date
df['booking_year'] = df['booking_date'].dt.year
```

```

df['booking_month'] = df['booking_date'].dt.month
df['booking_day'] = df['booking_date'].dt.day
df['booking_weekday'] = df['booking_date'].dt.dayofweek

# Ship Date
df['ship_year'] = df['ship_date'].dt.year
df['ship_month'] = df['ship_date'].dt.month
df['ship_day'] = df['ship_date'].dt.day
df['ship_weekday'] = df['ship_date'].dt.dayofweek

# Registration
df['registration_year'] = df['registration_date'].dt.year

print("Feature Engineering Completed Successfully")

```

Feature Engineering Completed Successfully

```

# =====
# DATASET OPTIMIZATION TO REDUCE OVERFITTING
# (Run BEFORE creating X and y)
# =====

# 1. Remove less useful year columns
id_columns = [
    "booking_year",
    "ship_year",
    "registration_year"
]

df = df.drop(columns=id_columns, errors="ignore")

print("Step 1 Completed : Year columns removed")

# -----

# 2. Remove high-cardinality location columns
high_cardinality = [
    "headquarters",
    "city_x",
    "city_y",
    "city",
    "state_x",
    "state_y",
    "origin_city",
    "destination_city",
    "origin_country",
    "destination_country_x",
    "destination_country_y",
    "origin_port",
    "destination_port"
]

df = df.drop(columns=high_cardinality, errors="ignore")

print("Step 2 Completed : High-cardinality columns removed")

# -----

# 3. Remove constant columns
constant_columns = []

for col in df.columns:
    if df[col].nunique() <= 1:
        constant_columns.append(col)

print("\nConstant Columns Found :", constant_columns)

df = df.drop(columns=constant_columns, errors="ignore")

print("Step 3 Completed : Constant columns removed")

# -----

# 4. Remove Highly Correlated Features

```

```

# (Ignore target column)

corr_matrix = df.select_dtypes(include=['int64', 'int32', 'float64']).corr().abs()

upper_triangle = corr_matrix.where(
    np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
)

correlated_features = [
    column
    for column in upper_triangle.columns
    if any(upper_triangle[column] > 0.95)
]

print("Highly Correlated Features:")
print(correlated_features)

# -----

# 5. Check Remaining Dataset

print("\nFinal Dataset Shape :", df.shape)

print("\nRemaining Columns :", len(df.columns))

```

```

Step 1 Completed : Year columns removed
Step 2 Completed : High-cardinality columns removed

Constant Columns Found : []
Step 3 Completed : Constant columns removed
Highly Correlated Features:
['delay_days', 'ship_month']

Final Dataset Shape : (4020, 79)

Remaining Columns : 79

```

```

# Target Variable
y = df["shipment_delayed"]

# Columns to remove
drop_columns = [

    # Target & Leakage
    "shipment_delayed",
    "shipment_delay",
    "delay_days",
    "actual_delivery_date",
    "delivery_status",
    "delivery_attempts",

    # Original Date Columns
    "booking_date",
    "ship_date",
    "expected_delivery_date",
    "registration_date",
    "forecast_date",

    # IDs
    "shipment_id",
    "booking_id",
    "customer_id",
    "product_id",
    "carrier_id",
    "warehouse_id",
    "route_id",
    "vehicle_id",
    "customs_id",
    "weather_id",
    "history_id",

    # Names
    "customer_name",
    "carrier_name",
    "product_name",
    "supplier_name",
    "warehouse_name",

```

```
        "vehicle_number"
    ]

X = df.drop(columns=drop_columns, errors="ignore")

print("Feature Matrix Shape :", X.shape)
print("Target Shape :", y.shape)
```

```
Feature Matrix Shape : (4020, 51)
Target Shape : (4020,)
```

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

categorical_columns = X.select_dtypes(include=["object"]).columns

for col in categorical_columns:
    X[col] = le.fit_transform(X[col].astype(str))

print("Encoding Completed Successfully")

print(X.dtypes.value_counts())
```

```
Encoding Completed Successfully
int64      20
float64    16
bool        9
int32       6
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training Features :", X_train.shape)
print("Testing Features  :", X_test.shape)
print("Training Target   :", y_train.shape)
print("Testing Target    :", y_test.shape)
```

```
Training Features : (3216, 51)
Testing Features  : (804, 51)
Training Target   : (3216,)
Testing Target    : (804,)
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

numeric_columns = X_train.select_dtypes(
    include=["int64", "int32", "float64"]
).columns

X_train[numeric_columns] = scaler.fit_transform(
    X_train[numeric_columns]
)

X_test[numeric_columns] = scaler.transform(
    X_test[numeric_columns]
)

print("Standard Scaling Completed Successfully")
```

```
Standard Scaling Completed Successfully
```

```
from sklearn.linear_model import LogisticRegression

# Create Model
lr_model = LogisticRegression(
    random_state=42,
    max_iter=1000
)
```

```
# Train Model
lr_model.fit(X_train, y_train)

# Verify Model
print("="*60)
print("LOGISTIC REGRESSION MODEL")
print("="*60)
print("Model Type :", type(lr_model).__name__)
print("Training Completed :", hasattr(lr_model, "coef_"))
print("Number of Features :", X_train.shape[1])
print("Number of Classes :", len(lr_model.classes_))
print("Classes :", lr_model.classes_)

# Prediction
lr_train = lr_model.predict(X_train)
lr_test = lr_model.predict(X_test)

print("\nPrediction Completed Successfully")
```

```
=====
LOGISTIC REGRESSION MODEL
=====
Model Type : LogisticRegression
Training Completed : True
Number of Features : 51
Number of Classes : 2
Classes : [0 1]

Prediction Completed Successfully
```

```
print("="*60)
print("LOGISTIC REGRESSION RESULTS")
print("="*60)

print("Train Accuracy :", accuracy_score(y_train, lr_train))
print("Test Accuracy :", accuracy_score(y_test, lr_test))

print("\nTrain Precision :", precision_score(y_train, lr_train))
print("Train Recall :", recall_score(y_train, lr_train))
print("Train F1 Score :", f1_score(y_train, lr_train))

print("\nTest Precision :", precision_score(y_test, lr_test))
print("Test Recall :", recall_score(y_test, lr_test))
print("Test F1 Score :", f1_score(y_test, lr_test))

print("\nClassification Report\n")
print(classification_report(y_test, lr_test))
```

```
=====
LOGISTIC REGRESSION RESULTS
=====
Train Accuracy : 0.681592039800995
Test Accuracy : 0.6355721393034826

Train Precision : 0.6186830015313936
Train Recall : 0.3426632739609839
Train F1 Score : 0.4410480349344978

Test Precision : 0.5067567567567568
Test Recall : 0.2542372881355932
Test F1 Score : 0.33860045146726864

Classification Report
```

	precision	recall	f1-score	support
0	0.66	0.86	0.75	509
1	0.51	0.25	0.34	295
accuracy			0.64	804
macro avg	0.59	0.56	0.54	804
weighted avg	0.61	0.64	0.60	804

```
cm_lr = confusion_matrix(y_test, lr_test)

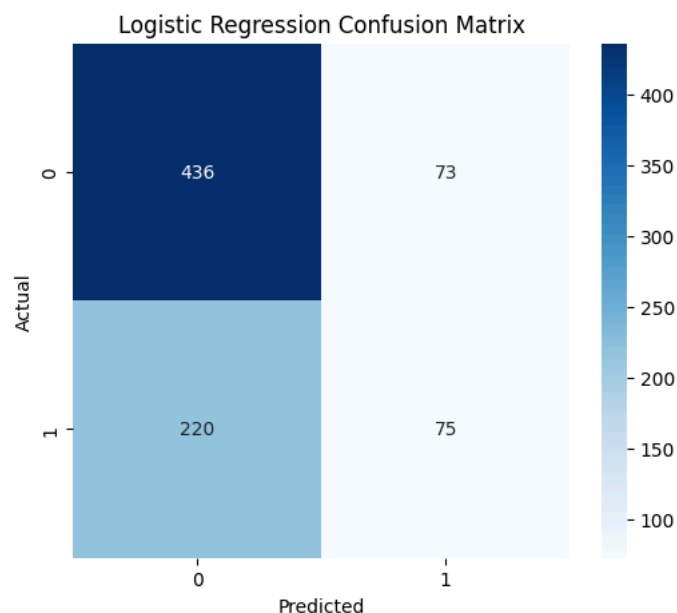
plt.figure(figsize=(6,5))
sns.heatmap(
```

```

cm_lr,
annot=True,
fmt="d",
cmap="Blues"
)

plt.title("Logistic Regression Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```



```

from sklearn.naive_bayes import GaussianNB

# Create Model
nb_model = GaussianNB()

# Train Model
nb_model.fit(X_train, y_train)

# Verify Model
print("="*60)
print("GAUSSIAN NAIVE BAYES MODEL")
print("="*60)
print("Model Type :", type(nb_model).__name__)
print("Training Completed :", hasattr(nb_model, "theta_")) # theta_ and sigma_ are attributes for trained GaussianNB
print("Number of Features :", X_train.shape[1])
print("Number of Classes :", len(nb_model.classes_))
print("Classes :", nb_model.classes_)

# Prediction
nb_train = nb_model.predict(X_train)
nb_test = nb_model.predict(X_test)

print("\nPrediction Completed Successfully")

```

```

=====
GAUSSIAN NAIVE BAYES MODEL
=====
Model Type : GaussianNB
Training Completed : True
Number of Features : 51
Number of Classes : 2
Classes : [0 1]

Prediction Completed Successfully

```

```

print("\n" + "="*60)
print("GAUSSIAN NAIVE BAYES RESULTS")
print("="*60)

print("Train Accuracy :", accuracy_score(y_train, nb_train))

```

```

print("Test Accuracy :", accuracy_score(y_test, nb_test))

print("\nTrain Precision :", precision_score(y_train, nb_train))
print("Train Recall :", recall_score(y_train, nb_train))
print("Train F1 Score :", f1_score(y_train, nb_train))

print("\nTest Precision :", precision_score(y_test, nb_test))
print("Test Recall :", recall_score(y_test, nb_test))
print("Test F1 Score :", f1_score(y_test, nb_test))

print("\nClassification Report\n")
print(classification_report(y_test, nb_test))

```

```

=====
GAUSSIAN NAIVE BAYES RESULTS
=====
Train Accuracy : 0.6672885572139303
Test Accuracy : 0.6629353233830846

Train Precision : 0.5620022753128555
Train Recall : 0.41899915182357933
Train F1 Score : 0.4800777453838678

Test Precision : 0.5674157303370787
Test Recall : 0.3423728813559322
Test F1 Score : 0.427061310782241

Classification Report

```

	precision	recall	f1-score	support
0	0.69	0.85	0.76	509
1	0.57	0.34	0.43	295
accuracy			0.66	804
macro avg	0.63	0.60	0.59	804
weighted avg	0.65	0.66	0.64	804

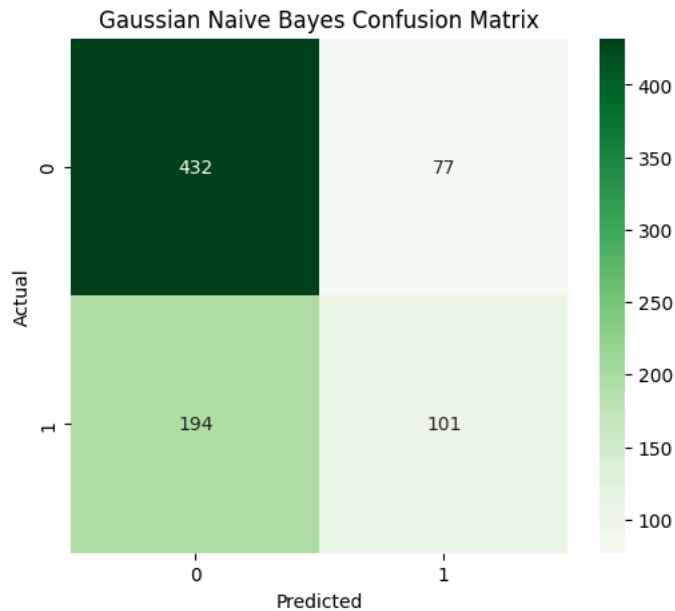
```

cm_nb = confusion_matrix(y_test, nb_test)

plt.figure(figsize=(6,5))
sns.heatmap(
    cm_nb,
    annot=True,
    fmt="d",
    cmap="Greens"
)

plt.title("Gaussian Naive Bayes Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```

```
from sklearn.neighbors import KNeighborsClassifier

# Create Model
knn_model = KNeighborsClassifier()

# Train Model
knn_model.fit(X_train, y_train)

# Verify Model
print("="*60)
print("K-NEAREST NEIGHBORS MODEL")
print("="*60)
print("Model Type :", type(knn_model).__name__)
print("Training Completed :", hasattr(knn_model, "_fit_X"))
print("Number of Features :", X_train.shape[1])
print("Number of Neighbors :", knn_model.n_neighbors)

# Prediction
knn_train = knn_model.predict(X_train)
knn_test = knn_model.predict(X_test)

print("\nPrediction Completed Successfully")
```

```
=====
K-NEAREST NEIGHBORS MODEL
=====
Model Type : KNeighborsClassifier
Training Completed : True
Number of Features : 51
Number of Neighbors : 5

Prediction Completed Successfully
```

```
print("\n" + "="*60)
print("K-NEAREST NEIGHBORS RESULTS")
print("="*60)

print("Train Accuracy :", accuracy_score(y_train, knn_train))
print("Test Accuracy :", accuracy_score(y_test, knn_test))

print("\nTrain Precision :", precision_score(y_train, knn_train))
print("Train Recall :", recall_score(y_train, knn_train))
print("Train F1 Score :", f1_score(y_train, knn_train))

print("\nTest Precision :", precision_score(y_test, knn_test))
print("Test Recall :", recall_score(y_test, knn_test))
print("Test F1 Score :", f1_score(y_test, knn_test))

print("\nClassification Report\n")
print(classification_report(y_test, knn_test))
```

```
=====
K-NEAREST NEIGHBORS RESULTS
=====
Train Accuracy : 0.726679104477612
Test Accuracy : 0.599502487562189

Train Precision : 0.7021563342318059
Train Recall : 0.44189991518235794
Train F1 Score : 0.5424258198854763

Test Precision : 0.41916167664670656
Test Recall : 0.23728813559322035
Test F1 Score : 0.30303030303030304
```

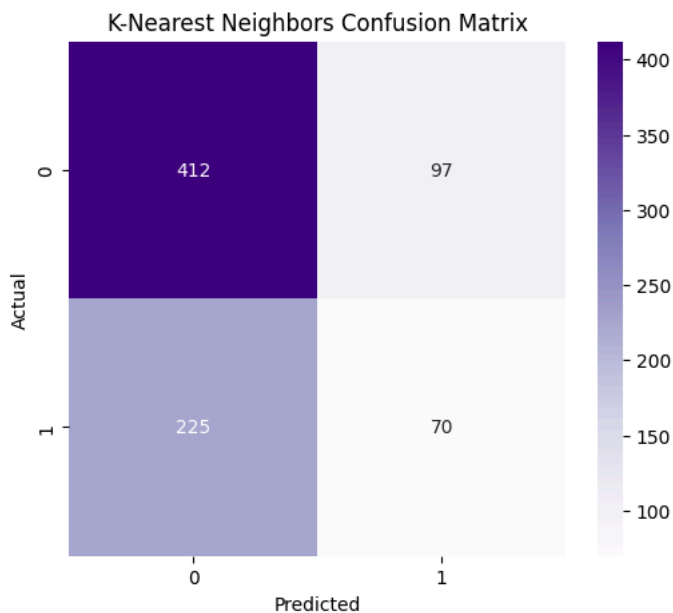
Classification Report

	precision	recall	f1-score	support
0	0.65	0.81	0.72	509
1	0.42	0.24	0.30	295
accuracy			0.60	804
macro avg	0.53	0.52	0.51	804
weighted avg	0.56	0.60	0.57	804

```
cm_knn = confusion_matrix(y_test, knn_test)

plt.figure(figsize=(6,5))
sns.heatmap(
    cm_knn,
    annot=True,
    fmt="d",
    cmap="Purples"
)

plt.title("K-Nearest Neighbors Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```
from sklearn.tree import DecisionTreeClassifier

# Create Model
dt_model = DecisionTreeClassifier(
    max_depth=8,
    min_samples_leaf=5,
    random_state=42
)

# Train Model
```

```
dt_model.fit(X_train, y_train)

# Verify Model
print("="*60)
print("DECISION TREE MODEL")
print("="*60)
print("Model Type :", type(dt_model).__name__)
print("Training Completed :", hasattr(dt_model, "tree_"))
print("Number of Features :", X_train.shape[1])
print("Max Depth :", dt_model.max_depth)
print("Min Samples Leaf :", dt_model.min_samples_leaf)

# Prediction
dt_train = dt_model.predict(X_train)
dt_test = dt_model.predict(X_test)

print("\nPrediction Completed Successfully")
```

```
=====
DECISION TREE MODEL
=====
Model Type : DecisionTreeClassifier
Training Completed : True
Number of Features : 51
Max Depth : 8
Min Samples Leaf : 5

Prediction Completed Successfully
```

```
print("\n" + "="*60)
print("DECISION TREE RESULTS")
print("="*60)

print("Train Accuracy :", accuracy_score(y_train, dt_train))
print("Test Accuracy :", accuracy_score(y_test, dt_test))

print("\nTrain Precision :", precision_score(y_train, dt_train))
print("Train Recall :", recall_score(y_train, dt_train))
print("Train F1 Score :", f1_score(y_train, dt_train))

print("\nTest Precision :", precision_score(y_test, dt_test))
print("Test Recall :", recall_score(y_test, dt_test))
print("Test F1 Score :", f1_score(y_test, dt_test))

print("\nClassification Report\n")
print(classification_report(y_test, dt_test))
```

```
=====
DECISION TREE RESULTS
=====
Train Accuracy : 0.783271144278607
Test Accuracy : 0.6430348258706468

Train Precision : 0.7931873479318735
Train Recall : 0.553011026293469
Train F1 Score : 0.6516741629185407

Test Precision : 0.521505376344086
Test Recall : 0.328813593220339
Test F1 Score : 0.40332640332640335

Classification Report
```

	precision	recall	f1-score	support
0	0.68	0.83	0.75	509
1	0.52	0.33	0.40	295
accuracy			0.64	804
macro avg	0.60	0.58	0.57	804
weighted avg	0.62	0.64	0.62	804

```
cm_dt = confusion_matrix(y_test, dt_test)

plt.figure(figsize=(6,5))
sns.heatmap(
    cm_dt,
```

```

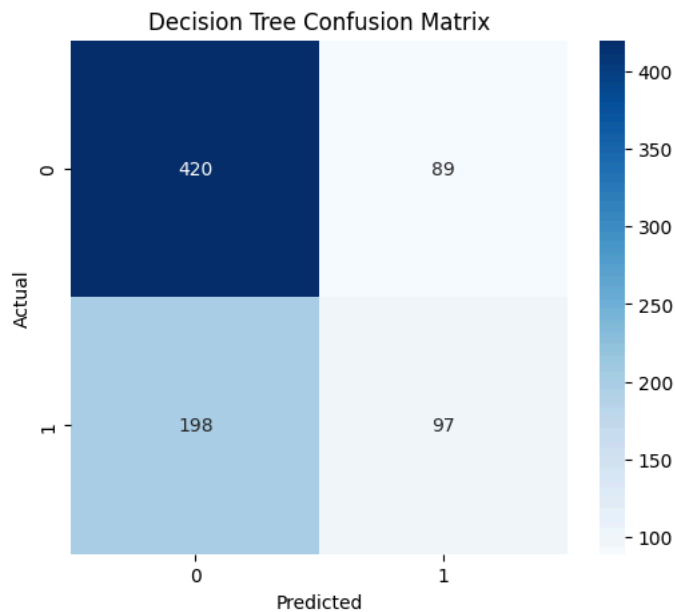
annot=True,
fmt="d",
cmap="Blues"
)

```

```

plt.title("Decision Tree Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```



```

from sklearn.ensemble import RandomForestClassifier

# Create Model
rf_model = RandomForestClassifier(
    max_depth=8,
    min_samples_leaf=5,
    random_state=42
)

# Train Model
rf_model.fit(X_train, y_train)

# Verify Model
print("="*60)
print("RANDOM FOREST MODEL")
print("="*60)
print("Model Type :", type(rf_model).__name__)
print("Training Completed :", hasattr(rf_model, "estimators_"))
print("Number of Features :", X_train.shape[1])
print("Max Depth :", rf_model.max_depth)
print("Min Samples Leaf :", rf_model.min_samples_leaf)

# Prediction
rf_train = rf_model.predict(X_train)
rf_test = rf_model.predict(X_test)

print("\nPrediction Completed Successfully")

```

```

=====
RANDOM FOREST MODEL
=====
Model Type : RandomForestClassifier
Training Completed : True
Number of Features : 51
Max Depth : 8
Min Samples Leaf : 5

Prediction Completed Successfully

```

```

print("\n" + "="*60)
print("RANDOM FOREST RESULTS (with max_depth=8, min_samples_leaf=5)")
print("="*60)

```

```

print("\n")

print("Train Accuracy :", accuracy_score(y_train, rf_train))
print(
    "Test Accuracy :",
    accuracy_score(y_test, rf_test)
)

print("\nTrain Precision :", precision_score(y_train, rf_train))
print("Train Recall :", recall_score(y_train, rf_train))
print("Train F1 Score :", f1_score(y_train, rf_train))

print("\nTest Precision :", precision_score(y_test, rf_test))
print("Test Recall :", recall_score(y_test, rf_test))
print("Test F1 Score :", f1_score(y_test, rf_test))

print("\nClassification Report\n")
print(classification_report(y_test, rf_test))

```

```

=====
RANDOM FOREST RESULTS (with max_depth=8, min_samples_leaf=5)
=====
Train Accuracy : 0.8006840796019901
Test Accuracy : 0.6666666666666666

Train Precision : 0.9873188405797102
Train Recall : 0.46225614927905007
Train F1 Score : 0.6296938186019642

Test Precision : 0.6391752577319587
Test Recall : 0.21016949152542372
Test F1 Score : 0.3163265306122449

```

Classification Report

	precision	recall	f1-score	support
0	0.67	0.93	0.78	509
1	0.64	0.21	0.32	295
accuracy			0.67	804
macro avg	0.65	0.57	0.55	804
weighted avg	0.66	0.67	0.61	804

```

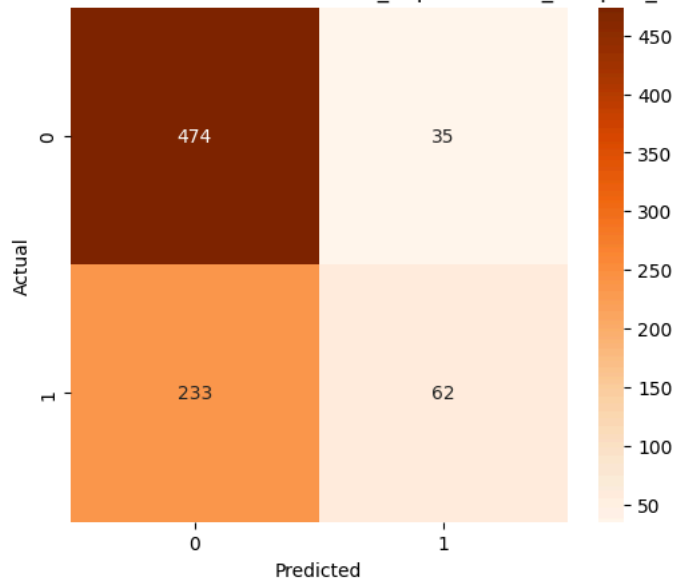
cm_rf = confusion_matrix(y_test, rf_test)

plt.figure(figsize=(6,5))
sns.heatmap(
    cm_rf,
    annot=True,
    fmt="d",
    cmap="Oranges"
)

plt.title("Random Forest Confusion Matrix (with max_depth=8, min_samples_leaf=5)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```

Random Forest Confusion Matrix (with max_depth=8, min_samples_leaf=5)



```

from sklearn.svm import SVC
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, classification_report, confusion_matrix)
import matplotlib.pyplot as plt
import seaborn as sns

# Create Model
svm_model = SVC(random_state=42)

# Train Model
svm_model.fit(X_train, y_train)

# Verify Model
print("="*60)
print("SUPPORT VECTOR MACHINE MODEL")
print("="*60)
print("Model Type :", type(svm_model).__name__)
print("Training Completed :", hasattr(svm_model, "n_features_in_"))
print("Number of Features :", X_train.shape[1])

# Prediction
svm_train = svm_model.predict(X_train)
svm_test = svm_model.predict(X_test)

print("\nPrediction Completed Successfully")

```

```

=====
SUPPORT VECTOR MACHINE MODEL
=====
Model Type : SVC
Training Completed : True
Number of Features : 51

Prediction Completed Successfully

```

```

# Evaluation
print("\n" + "="*60)
print("SUPPORT VECTOR MACHINE RESULTS")
print("="*60)

print("Train Accuracy :", accuracy_score(y_train, svm_train))
print("Test Accuracy :", accuracy_score(y_test, svm_test))

print("\nTrain Precision :", precision_score(y_train, svm_train))
print("Train Recall :", recall_score(y_train, svm_train))
print("Train F1 Score :", f1_score(y_train, svm_train))

print("\nTest Precision :", precision_score(y_test, svm_test))
print("Test Recall :", recall_score(y_test, svm_test))
print("Test F1 Score :", f1_score(y_test, svm_test))

```

```
print("\nClassification Report\n")
print(classification_report(y_test, svm_test))
```

```
=====
SUPPORT VECTOR MACHINE RESULTS
=====
Train Accuracy : 0.8075248756218906
Test Accuracy : 0.6579601990049752

Train Precision : 0.93343653250774
Train Recall : 0.5114503816793893
Train F1 Score : 0.6608219178082192

Test Precision : 0.5847457627118644
Test Recall : 0.23389830508474577
Test F1 Score : 0.3341404358353511
```

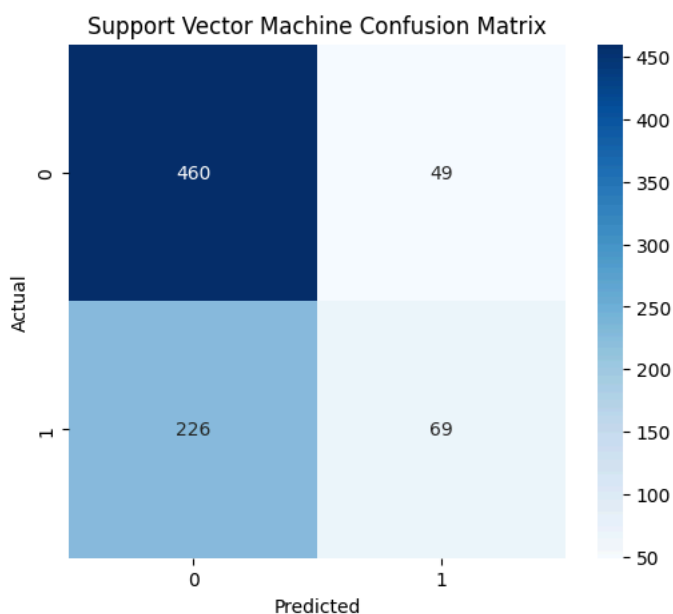
Classification Report

	precision	recall	f1-score	support
0	0.67	0.90	0.77	509
1	0.58	0.23	0.33	295
accuracy			0.66	804
macro avg	0.63	0.57	0.55	804
weighted avg	0.64	0.66	0.61	804

```
# Confusion Matrix
cm_svm = confusion_matrix(y_test, svm_test)

plt.figure(figsize=(6,5))
sns.heatmap(
    cm_svm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.title("Support Vector Machine Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```
from xgboost import XGBClassifier

# Create Model
xgb_model = XGBClassifier(random_state=42, use_label_encoder=False, eval_metric='logloss', max_depth=2, min_child_weight=10)

# Train Model
```

```
xgb_model.fit(X_train, y_train)

# Verify Model
print("="*60)
print("XGBOOST MODEL")
print("="*60)
print("Model Type :", type(xgb_model).__name__)
print("Training Completed :", hasattr(xgb_model, "feature_importances_"))
print("Number of Features :", X_train.shape[1])
print("Max Depth :", xgb_model.max_depth)
print("Min Child Weight :", xgb_model.min_child_weight)

# Prediction
xgb_train = xgb_model.predict(X_train)
xgb_test = xgb_model.predict(X_test)

print("\nPrediction Completed Successfully")
```

```
=====
XGBOOST MODEL
=====
Model Type : XGBClassifier
Training Completed : True
Number of Features : 51
Max Depth : 2
Min Child Weight : 10

Prediction Completed Successfully
```

```
print("\n" + "="*60)
print("XGBOOST RESULTS")
print("="*60)

print("Train Accuracy :", accuracy_score(y_train, xgb_train))
print("Test Accuracy :", accuracy_score(y_test, xgb_test))

print("\nTrain Precision :", precision_score(y_train, xgb_train))
print("Train Recall :", recall_score(y_train, xgb_train))
print("Train F1 Score :", f1_score(y_train, xgb_train))

print("\nTest Precision :", precision_score(y_test, xgb_test))
print("Test Recall :", recall_score(y_test, xgb_test))
print("Test F1 Score :", f1_score(y_test, xgb_test))

print("\nClassification Report\n")
print(classification_report(y_test, xgb_test))
```

```
=====
XGBOOST RESULTS
=====
Train Accuracy : 0.7748756218905473
Test Accuracy : 0.6728855721393034

Train Precision : 0.7905491698595147
Train Recall : 0.5250212044105174
Train F1 Score : 0.6309887869520897

Test Precision : 0.5784313725490197
Test Recall : 0.4
Test F1 Score : 0.4729458917835671

Classification Report
```

	precision	recall	f1-score	support
0	0.70	0.83	0.76	509
1	0.58	0.40	0.47	295
accuracy			0.67	804
macro avg	0.64	0.62	0.62	804
weighted avg	0.66	0.67	0.66	804

```
# Confusion Matrix
cm_xgb = confusion_matrix(y_test, xgb_test)

plt.figure(figsize=(6,5))
sns.heatmap(
```

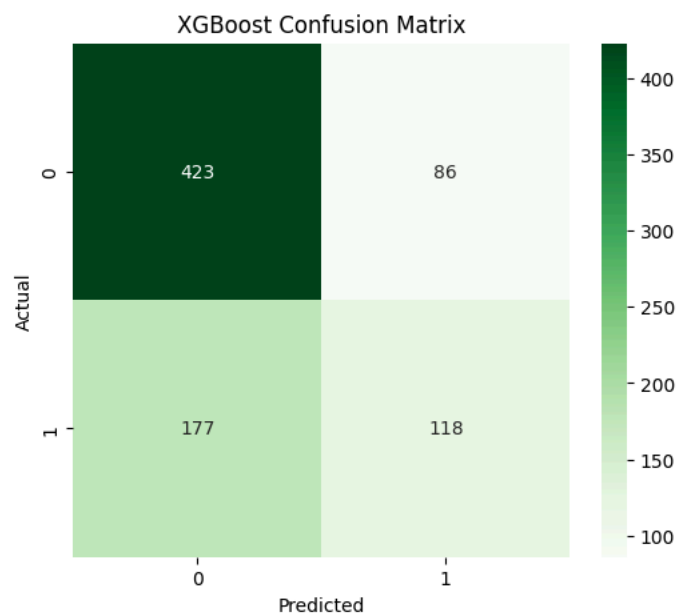


```

cm_xgb,
annot=True,
fmt="d",
cmap="Greens"
)

plt.title("XGBoost Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```



```

from catboost import CatBoostClassifier

# Create Model
cat_model = CatBoostClassifier(random_state=42, verbose=0, iterations=100) # Reduced iterations for faster execution, verbose=0

# Train Model
cat_model.fit(X_train, y_train)

# Verify Model
print("="*60)
print("CATBOOST MODEL")
print("="*60)
print("Model Type :", type(cat_model).__name__)
print("Training Completed :", hasattr(cat_model, "feature_importances_"))
print("Number of Features :", X_train.shape[1])
print("Number of Estimators :", cat_model.get_param('iterations'))

# Prediction
cat_train = cat_model.predict(X_train)
cat_test = cat_model.predict(X_test)

print("\nPrediction Completed Successfully")

```

```

=====
CATBOOST MODEL
=====
Model Type : CatBoostClassifier
Training Completed : True
Number of Features : 51
Number of Estimators : 100

Prediction Completed Successfully

```

```

print("\n" + "="*60)
print("CATBOOST RESULTS")
print("="*60)

print("Train Accuracy :", accuracy_score(y_train, cat_train))
print("Test Accuracy :", accuracy_score(y_test, cat_test))

```

```

print("\nTrain Precision :", precision_score(y_train, cat_train))
print("Train Recall :", recall_score(y_train, cat_train))
print("Train F1 Score :", f1_score(y_train, cat_train))

print("\nTest Precision :", precision_score(y_test, cat_test))
print("Test Recall :", recall_score(y_test, cat_test))
print("Test F1 Score :", f1_score(y_test, cat_test))

print("\nClassification Report\n")
print(classification_report(y_test, cat_test))

```

```

=====
CATBOOST RESULTS
=====
Train Accuracy : 0.9107587064676617
Test Accuracy : 0.6666666666666666

Train Precision : 0.9795698924731183
Train Recall : 0.7726887192536047
Train F1 Score : 0.8639165481270744

Test Precision : 0.574585635359116
Test Recall : 0.3525423728813559
Test F1 Score : 0.4369747899159664

Classification Report

```

	precision	recall	f1-score	support
0	0.69	0.85	0.76	509
1	0.57	0.35	0.44	295
accuracy			0.67	804
macro avg	0.63	0.60	0.60	804
weighted avg	0.65	0.67	0.64	804

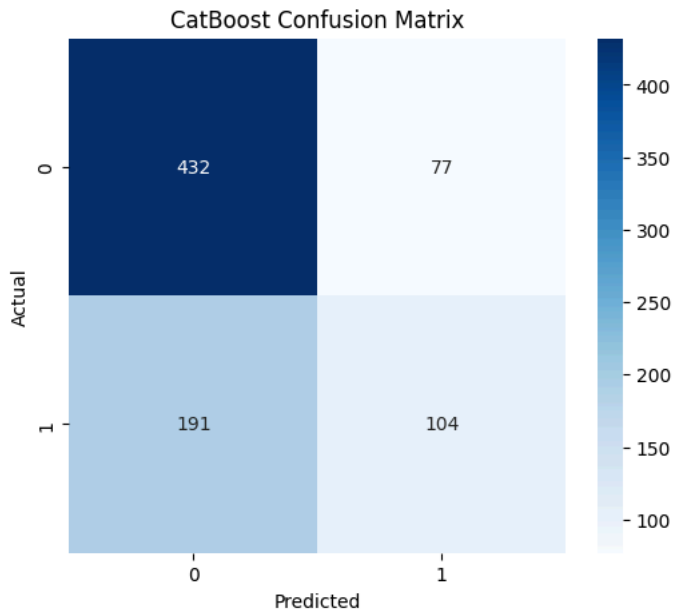
```

# Confusion Matrix
cm_cat = confusion_matrix(y_test, cat_test)

plt.figure(figsize=(6,5))
sns.heatmap(
    cm_cat,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.title("CatBoost Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```



```
!pip install catboost
```

```
Requirement already satisfied: catboost in /usr/local/lib/python3.12/dist-packages (1.2.10)
Requirement already satisfied: graphviz in /usr/local/lib/python3.12/dist-packages (from catboost) (0.21)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from catboost) (3.10.0)
Requirement already satisfied: numpy<3.0,>=1.16.0 in /usr/local/lib/python3.12/dist-packages (from catboost) (2.0.2)
Requirement already satisfied: pandas<4.0,>=0.24 in /usr/local/lib/python3.12/dist-packages (from catboost) (2.2.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from catboost) (1.16.3)
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (from catboost) (5.24.1)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from catboost) (1.17.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas<4.0,>=0.24->catboost) (2025.
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<4.0,>=0.24->catboost) (202
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (26.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (3.3.2)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/dist-packages (from plotly->catboost) (9.1.4)
```

```
model_names = [
    "Logistic Regression",
    "Gaussian Naive Bayes",
    "K-Nearest Neighbors",
    "Decision Tree",
    "Random Forest",
    "Support Vector Machine",
    "XGBoost",
    "CatBoost"
]

predictions = [
    lr_test,
    nb_test,
    knn_test,
    dt_test,
    rf_test,
    svm_test,
    xgb_test,
    cat_test
]

results = []
for name, pred in zip(model_names, predictions):
    accuracy = accuracy_score(y_test, pred)
    precision = precision_score(y_test, pred)
    recall = recall_score(y_test, pred)
    f1 = f1_score(y_test, pred)
```

```

    results.append({
        "Model": name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1
    })

results_df = pd.DataFrame(results)
results_df = results_df.sort_values(by="F1-Score", ascending=False).reset_index(drop=True)

print("\n" + "="*60)
print("MODEL COMPARISON - TEST SET METRICS")
print("="*60)
print(results_df.to_string())

best_model_f1 = results_df.loc[results_df['F1-Score'].idxmax()]
best_model_accuracy = results_df.loc[results_df['Accuracy'].idxmax()]

print("\n" + "="*60)
print("BEST PERFORMING MODEL (based on F1-Score)")
print("="*60)
print(best_model_f1.to_string())

print("\nReasoning for selection (based on F1-Score):\n" \
      f"The {best_model_f1['Model']} model achieved the highest F1-Score of {best_model_f1['F1-Score']:.4f} \n" \
      "on the test set. F1-Score is a harmonic mean of precision and recall, making it a good metric \n" \
      "for classification tasks, especially when there might be imbalanced classes or when both \n" \
      "false positives and false negatives are important to consider. A higher F1-Score indicates \n" \
      "a better balance between correctly identifying positive cases (recall) and avoiding \n" \
      "incorrectly labeling negative cases as positive (precision).")

```

```

=====
MODEL COMPARISON - TEST SET METRICS
=====

```

	Model	Accuracy	Precision	Recall	F1-Score
0	XGBoost	0.672886	0.578431	0.400000	0.472946
1	CatBoost	0.666667	0.574586	0.352542	0.436975
2	Gaussian Naive Bayes	0.662935	0.567416	0.342373	0.427061
3	Decision Tree	0.643035	0.521505	0.328814	0.403326
4	Logistic Regression	0.635572	0.506757	0.254237	0.338600
5	Support Vector Machine	0.657960	0.584746	0.233898	0.334140
6	Random Forest	0.666667	0.639175	0.210169	0.316327
7	K-Nearest Neighbors	0.599502	0.419162	0.237288	0.303030

```

=====
BEST PERFORMING MODEL (based on F1-Score)
=====
Model          XGBoost
Accuracy       0.672886
Precision      0.578431
Recall         0.4
F1-Score       0.472946

```

Reasoning for selection (based on F1-Score):
 The XGBoost model achieved the highest F1-Score of 0.4729
 on the test set. F1-Score is a harmonic mean of precision and recall, making it a good metric
 for classification tasks, especially when there might be imbalanced classes or when both
 false positives and false negatives are important to consider. A higher F1-Score indicates
 a better balance between correctly identifying positive cases (recall) and avoiding
 incorrectly labeling negative cases as positive (precision).

Optimization

```

from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report
import numpy as np

print("""=====
HYPERPARAMETER TUNING
=====

```

```

""")

# 1. Logistic Regression Hyperparameter Tuning
print("Tuning Logistic Regression...")
param_grid_lr = {
    'C': [0.01, 0.1, 1.0, 10.0],
    'solver': ['liblinear', 'saga'],
    'penalty': ['l1', 'l2']
}
grid_search_lr = GridSearchCV(LogisticRegression(random_state=42, max_iter=1000), param_grid_lr, cv=3, scoring='f1', n_jobs=-1,
grid_search_lr.fit(X_train, y_train)
print(f"Best parameters for Logistic Regression: {grid_search_lr.best_params_}")
print(f"Best F1-Score for Logistic Regression (Cross-Validation): {grid_search_lr.best_score_:.4f}")
best_lr_model = grid_search_lr.best_estimator_

# Evaluate best LR model on test set
y_pred_lr = best_lr_model.predict(X_test)
print(f"Test Accuracy (LR): {accuracy_score(y_test, y_pred_lr):.4f}")
print(f"Test Precision (LR): {precision_score(y_test, y_pred_lr):.4f}")
print(f"Test Recall (LR): {recall_score(y_test, y_pred_lr):.4f}")
print(f"Test F1-Score (LR): {f1_score(y_test, y_pred_lr):.4f}")
print("Classification Report (LR):\n", classification_report(y_test, y_pred_lr))
print("\n")

# 2. Gaussian Naive Bayes Hyperparameter Tuning
print("Tuning Gaussian Naive Bayes...")
param_grid_nb = {
    'var_smoothing': np.logspace(0, -9, num=100)
}
grid_search_nb = GridSearchCV(GaussianNB(), param_grid_nb, cv=3, scoring='f1', n_jobs=-1, verbose=1)
grid_search_nb.fit(X_train, y_train)
print(f"Best parameters for Gaussian Naive Bayes: {grid_search_nb.best_params_}")
print(f"Best F1-Score for Gaussian Naive Bayes (Cross-Validation): {grid_search_nb.best_score_:.4f}")
best_nb_model = grid_search_nb.best_estimator_

# Evaluate best NB model on test set
y_pred_nb = best_nb_model.predict(X_test)
print(f"Test Accuracy (NB): {accuracy_score(y_test, y_pred_nb):.4f}")
print(f"Test Precision (NB): {precision_score(y_test, y_pred_nb):.4f}")
print(f"Test Recall (NB): {recall_score(y_test, y_pred_nb):.4f}")
print(f"Test F1-Score (NB): {f1_score(y_test, y_pred_nb):.4f}")
print("Classification Report (NB):\n", classification_report(y_test, y_pred_nb))
print("\n")

# 3. Random Forest Hyperparameter Tuning
print("Tuning Random Forest...")
param_grid_rf = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 8, 10],
    'min_samples_leaf': [2, 4, 6]
}
grid_search_rf = GridSearchCV(RandomForestClassifier(random_state=42), param_grid_rf, cv=3, scoring='f1', n_jobs=-1, verbose=1)
grid_search_rf.fit(X_train, y_train)
print(f"Best parameters for Random Forest: {grid_search_rf.best_params_}")
print(f"Best F1-Score for Random Forest (Cross-Validation): {grid_search_rf.best_score_:.4f}")
best_rf_model = grid_search_rf.best_estimator_

# Evaluate best RF model on test set
y_pred_rf = best_rf_model.predict(X_test)
print(f"Test Accuracy (RF): {accuracy_score(y_test, y_pred_rf):.4f}")
print(f"Test Precision (RF): {precision_score(y_test, y_pred_rf):.4f}")
print(f"Test Recall (RF): {recall_score(y_test, y_pred_rf):.4f}")
print(f"Test F1-Score (RF): {f1_score(y_test, y_pred_rf):.4f}")
print("Classification Report (RF):\n", classification_report(y_test, y_pred_rf))
print("\n")

# 4. XGBoost Hyperparameter Tuning
print("Tuning XGBoost...")
param_grid_xgb = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.05, 0.1]
}
grid_search_xgb = GridSearchCV(XGBClassifier(random_state=42, use_label_encoder=False, eval_metric='logloss'), param_grid_xgb,
grid_search_xgb.fit(X_train, y_train)
print(f"Best parameters for XGBoost: {grid_search_xgb.best_params_}")
print(f"Best F1-Score for XGBoost (Cross-Validation): {grid_search_xgb.best_score_:.4f}")

```

```

best_xgb_model = grid_search_xgb.best_estimator_

# Evaluate best XGB model on test set
y_pred_xgb = best_xgb_model.predict(X_test)
print(f"Test Accuracy (XGB): {accuracy_score(y_test, y_pred_xgb):.4f}")
print(f"Test Precision (XGB): {precision_score(y_test, y_pred_xgb):.4f}")
print(f"Test Recall (XGB): {recall_score(y_test, y_pred_xgb):.4f}")
print(f"Test F1-Score (XGB): {f1_score(y_test, y_pred_xgb):.4f}")
print("Classification Report (XGB):\n", classification_report(y_test, y_pred_xgb))
print("\n")

print("""=====
HYPERPARAMETER TUNING COMPLETED
=====
""")

```

```

Test Recall (NB): 0.3424
Test F1-Score (NB): 0.4271
Classification Report (NB):

```

	precision	recall	f1-score	support
0	0.69	0.85	0.76	509
1	0.57	0.34	0.43	295
accuracy			0.66	804
macro avg	0.63	0.60	0.59	804
weighted avg	0.65	0.66	0.64	804

```

Tuning Random Forest...
Fitting 3 folds for each of 27 candidates, totalling 81 fits
Best parameters for Random Forest: {'max_depth': 10, 'min_samples_leaf': 2, 'n_estimators': 100}
Best F1-Score for Random Forest (Cross-Validation): 0.3780
Test Accuracy (RF): 0.6654
Test Precision (RF): 0.6275
Test Recall (RF): 0.2169
Test F1-Score (RF): 0.3224
Classification Report (RF):

```

	precision	recall	f1-score	support
0	0.67	0.93	0.78	509
1	0.63	0.22	0.32	295
accuracy			0.67	804
macro avg	0.65	0.57	0.55	804
weighted avg	0.65	0.67	0.61	804

```

Tuning XGBoost...
Fitting 3 folds for each of 27 candidates, totalling 81 fits
Best parameters for XGBoost: {'learning_rate': 0.05, 'max_depth': 7, 'n_estimators': 200}
Best F1-Score for XGBoost (Cross-Validation): 0.4648
Test Accuracy (XGB): 0.6604
Test Precision (XGB): 0.5567
Test Recall (XGB): 0.3661
Test F1-Score (XGB): 0.4417
Classification Report (XGB):

```

	precision	recall	f1-score	support
0	0.69	0.83	0.76	509
1	0.56	0.37	0.44	295
accuracy			0.66	804
macro avg	0.63	0.60	0.60	804
weighted avg	0.64	0.66	0.64	804

```

=====
HYPERPARAMETER TUNING COMPLETED
=====

```

```

from sklearn.feature_selection import RFECV
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold

```

```

rf = RandomForestClassifier(
    n_estimators=100,

```

```

        max_depth=20,
        min_samples_split=2,
        min_samples_leaf=1,
        random_state=42,
        n_jobs=-1
    )

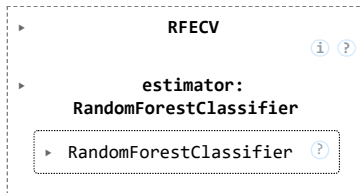
```

```

rfecv = RFECV(
    estimator=rf,
    step=1,
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    scoring='accuracy',
    n_jobs=-1,
    min_features_to_select=10
)

rfecv.fit(X_train, y_train)

```



```
print("Optimal Number of Features:", rfecv.n_features_)
```

```
Optimal Number of Features: 45
```

```

selected_features = X_train.columns[rfecv.support_]

print("\nSelected Features:")
for feature in selected_features:
    print(feature)

```

```

Selected Features:
shipping_mode
shipment_type
priority
weight_kg
volume_cbm
declared_value
insurance
fragile_x
carrier_type
average_rating
fleet_size
years_of_service
customer_type
industry
country
customs_required
documentation_complete
cargo_type
category
hs_code
weight_per_unit
distance_km
average_transit_days
route_risk
traffic_index
vehicle_type
capacity_kg
fuel_type
maintenance_status
vehicle_age
warehouse_capacity
current_utilization
warehouse_type
weather_condition
temperature
rainfall
humidity
wind_speed
visibility
booking_month

```

```

booking_day
booking_weekday
ship_month
ship_day
ship_weekday

```

```
removed_features = X_train.columns[~rfecv.support_]
```

```

print("\nRemoved Features:")
for feature in removed_features:
    print(feature)

```

```

Removed Features:
customer_status
inspection_required
hazardous
perishable
temperature_controlled
fragile_y

```

```

X_train_selected = X_train[selected_features]
X_test_selected = X_test[selected_features]

```

```

rf.fit(X_train_selected, y_train)

y_pred = rf.predict(X_test_selected)

```

```

from sklearn.metrics import accuracy_score, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

```

```

Accuracy: 0.6890547263681592

```

	precision	recall	f1-score	support
0	0.69	0.92	0.79	509
1	0.67	0.29	0.41	295
accuracy			0.69	804
macro avg	0.68	0.61	0.60	804
weighted avg	0.69	0.69	0.65	804

```

from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, classification_report, Conf
import matplotlib.pyplot as plt

# --- Random Forest Hyperparameter Tuning ---
print("--- Random Forest Hyperparameter Tuning (CV=5) ---")
rf_param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

rf_grid_search = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42, n_jobs=-1),
    param_grid=rf_param_grid,
    scoring='f1', # Optimize for F1-score
    cv=5, # 5-fold cross-validation as requested
    verbose=1,
    n_jobs=-1
)

rf_grid_search.fit(X_train_selected, y_train)

print("\nBest parameters for Random Forest:", rf_grid_search.best_params_)
print("Best F1 Score for Random Forest:", rf_grid_search.best_score_)

# Evaluate the best Random Forest model

```



```

best_rf = rf_grid_search.best_estimator_
y_pred_rf_tuned = best_rf.predict(X_test_selected)
y_prob_rf_tuned = best_rf.predict_proba(X_test_selected)[: , 1] # Probabilities for ROC AUC

print("\n--- Tuned Random Forest Classifier Performance (CV=5) ---")
print(f"Accuracy : {accuracy_score(y_test, y_pred_rf_tuned):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_rf_tuned):.4f}")
print(f"Recall : {recall_score(y_test, y_pred_rf_tuned):.4f}")
print(f"F1 Score : {f1_score(y_test, y_pred_rf_tuned):.4f}")
print(f"ROC AUC : {roc_auc_score(y_test, y_prob_rf_tuned):.4f}")
print("\nClassification Report (Tuned Random Forest):")
print(classification_report(y_test, y_pred_rf_tuned))
cm_rf_tuned = ConfusionMatrixDisplay.from_estimator(best_rf, X_test_selected, y_test, cmap=plt.cm.Blues)
plt.title('Confusion Matrix for Tuned Random Forest (CV=5)')
plt.show()

# --- XGBoost Hyperparameter Tuning ---
print("\n--- XGBoost Hyperparameter Tuning (CV=5) ---")
xgb_param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [3, 5],
    'learning_rate': [0.01, 0.1],
    'subsample': [0.8, 1.0]
}

xgb_grid_search = GridSearchCV(
    estimator=XGBClassifier(objective='binary:logistic', eval_metric='logloss', use_label_encoder=False, random_state=42, n_jobs
    param_grid=xgb_param_grid,
    scoring='f1',
    cv=5, # 5-fold cross-validation as requested
    verbose=1,
    n_jobs=-1
)

xgb_grid_search.fit(X_train_selected, y_train)

print("\nBest parameters for XGBoost:", xgb_grid_search.best_params_)
print("Best F1 Score for XGBoost:", xgb_grid_search.best_score_)

# Evaluate the best XGBoost model
best_xgb = xgb_grid_search.best_estimator_
y_pred_xgb_tuned = best_xgb.predict(X_test_selected)
y_prob_xgb_tuned = best_xgb.predict_proba(X_test_selected)[: , 1] # Probabilities for ROC AUC

print("\n--- Tuned XGBoost Classifier Performance (CV=5) ---")
print(f"Accuracy : {accuracy_score(y_test, y_pred_xgb_tuned):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_xgb_tuned):.4f}")
print(f"Recall : {recall_score(y_test, y_pred_xgb_tuned):.4f}")
print(f"F1 Score : {f1_score(y_test, y_pred_xgb_tuned):.4f}")
print(f"ROC AUC : {roc_auc_score(y_test, y_prob_xgb_tuned):.4f}")
print("\nClassification Report (Tuned XGBoost):")
print(classification_report(y_test, y_pred_xgb_tuned))
cm_xgb_tuned = ConfusionMatrixDisplay.from_estimator(best_xgb, X_test_selected, y_test, cmap=plt.cm.Blues)
plt.title('Confusion Matrix for Tuned XGBoost (CV=5)')
plt.show()

# --- Logistic Regression Hyperparameter Tuning ---
print("\n--- Logistic Regression Hyperparameter Tuning (CV=5) ---")

# Scale the data for Logistic Regression
scaler = StandardScaler()
X_train_lr_scaled = scaler.fit_transform(X_train_selected)
X_test_lr_scaled = scaler.transform(X_test_selected)

lr_param_grid = {
    'C': [0.1, 1.0, 10.0],
    'solver': ['liblinear'], # 'lbfgs' does not support 'l1' penalty with default settings
    'penalty': ['l1', 'l2']
}

lr_grid_search = GridSearchCV(
    estimator=LogisticRegression(random_state=42, max_iter=1000, n_jobs=-1),
    param_grid=lr_param_grid,
    scoring='f1',
    cv=5, # 5-fold cross-validation as requested

```

```
        verbose=1,
        n_jobs=-1
    )

lr_grid_search.fit(X_train_lr_scaled, y_train)

print("\nBest parameters for Logistic Regression:", lr_grid_search.best_params_)
print("Best F1 Score for Logistic Regression:", lr_grid_search.best_score_)

# Evaluate the best Logistic Regression model
best_lr = lr_grid_search.best_estimator_
y_pred_lr_tuned = best_lr.predict(X_test_lr_scaled)
y_prob_lr_tuned = best_lr.predict_proba(X_test_lr_scaled)[:, 1] # Probabilities for ROC AUC

print("\n--- Tuned Logistic Regression Classifier Performance (CV=5) ---")
print(f"Accuracy : {accuracy_score(y_test, y_pred_lr_tuned):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_lr_tuned):.4f}")
print(f"Recall   : {recall_score(y_test, y_pred_lr_tuned):.4f}")
print(f"F1 Score : {f1_score(y_test, y_pred_lr_tuned):.4f}")
print(f"ROC AUC  : {roc_auc_score(y_test, y_prob_lr_tuned):.4f}")
print("\nClassification Report (Tuned Logistic Regression):")
print(classification_report(y_test, y_pred_lr_tuned))
cm_lr_tuned = ConfusionMatrixDisplay.from_estimator(best_lr, X_test_lr_scaled, y_test, cmap=plt.cm.Blues)
plt.title('Confusion Matrix for Tuned Logistic Regression (CV=5)')
plt.show()
```